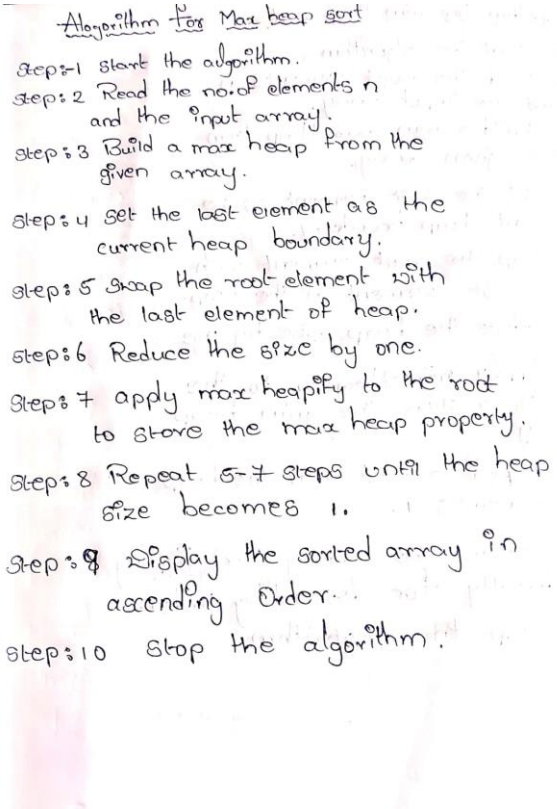
PRACTICAL 3

MAX HEAP SORT

AIM:

To implementation of max heap and min heap sort algorithm.

ALGORITHM:


Algorithm for Max heap sort

Step:1 Start the algorithm.
Step:2 Read the no. of elements n and the input array.
Step:3 Build a max heap from the given array.
Step:4 Set the last element as the current heap boundary.
Step:5 Swap the root element with the last element of heap.
Step:6 Reduce the size by one.
Step:7 apply max heapify to the root to store the max heap property.
Step:8 Repeat 5-7 steps until the heap size becomes 1.
Step:9 Display the sorted array in ascending Order.
Step:10 Stop the algorithm.

CODE:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

OUTPUT:

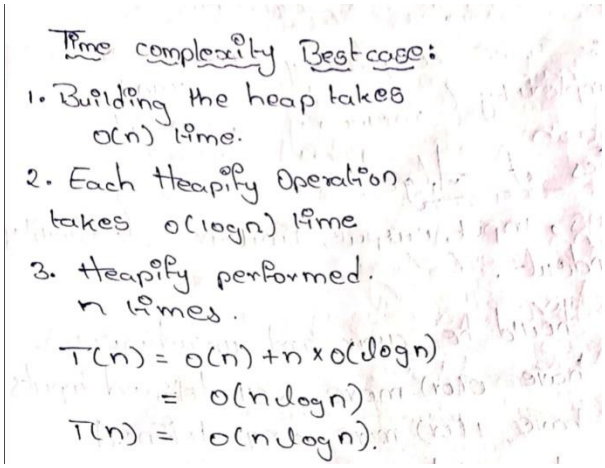
```
Output

Enter number of elements: 4
Enter elements: 10 20 30 40
Sorted array: 10 20 30 40

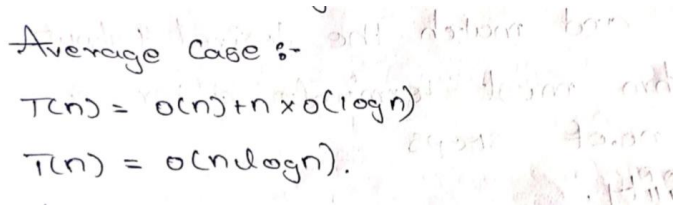
=== Code Execution Successful ===
```

TIME COMPLEXITY:

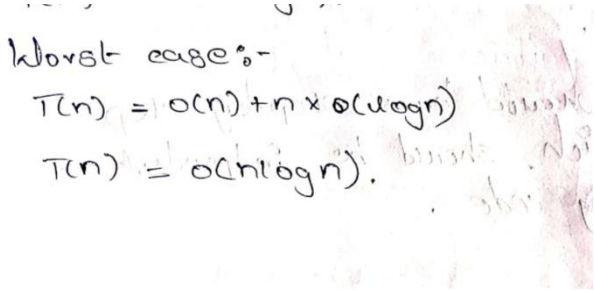
BEST CASE:



AVERAGE CASE:

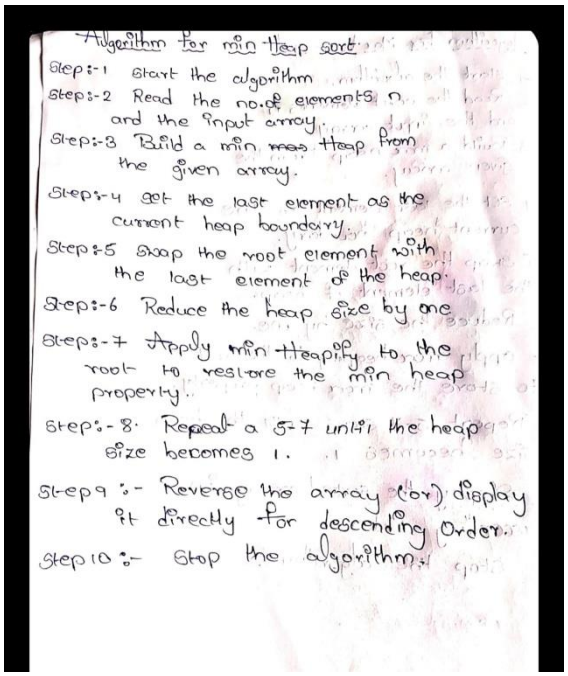


WORST CASE:



MIN HEAP SORT

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
   int smallest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
```

```
    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i)
    {
        swap(arr[i], arr[smallest]);
        heapify(arr, n, smallest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array in descending order: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
```

```
    return 0;
}
```

OUTPUT:

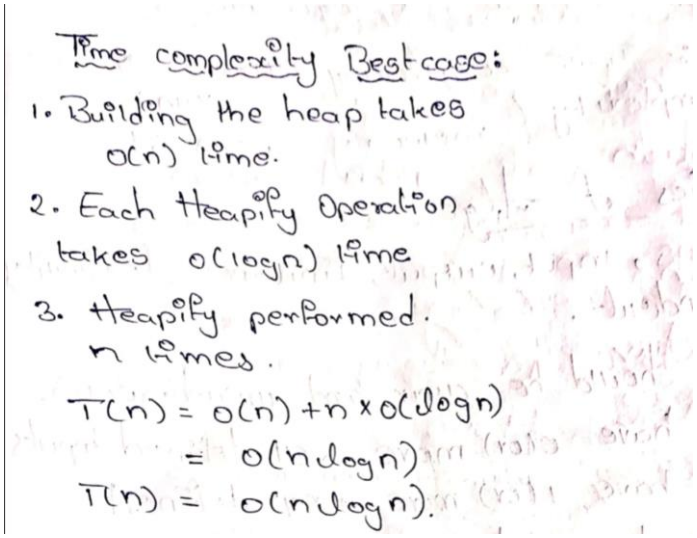
```
Output

Enter number of elements: 6
Enter elements: 20 30 40 50 60 80
Sorted array in descending order: 80 60 50 40 30 20

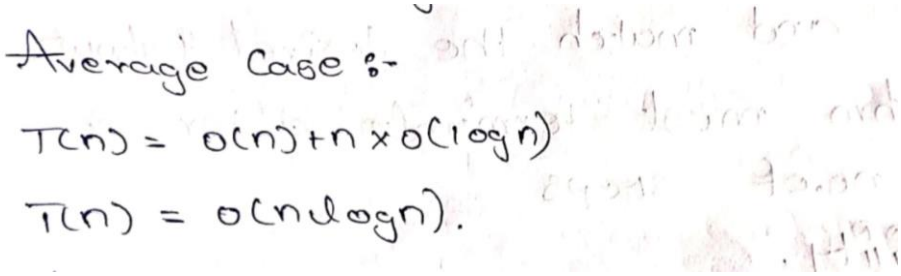
=== Code Execution Successful ===
```

TIMECOMPLEXITY:

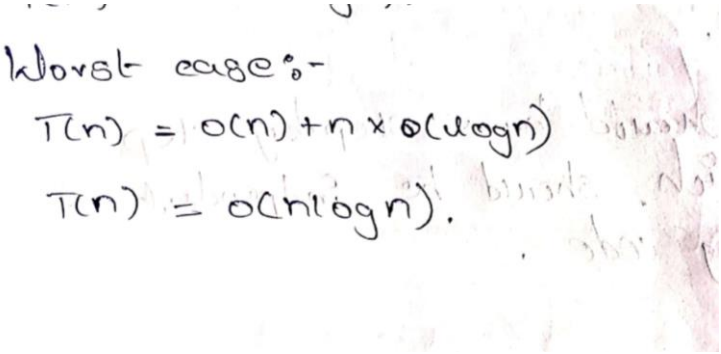
BEST CASE:



AVERAGE CASE:



WORST CASE:



CONCLUSION:

Both Max Heap Sort and Min Heap Sort are efficient sorting algorithms based on the heap data structure. Max Heap Sort produces the elements in ascending order, while Min Heap Sort produces the elements in descending order. Both algorithms have a time complexity of O(n log n) and require O(1) extra space. Therefore, heap sort is useful for sorting large amounts of data efficiently.